IoT Network Anomaly Detection Using Machine Learning

Submitted By

Student Name	Student ID
Shoaib Hossain	221-15-5949



DAFFODIL INTERNATIONAL UNIVERSITY

Dhaka, Bangladesh

April 14, 2025

DECLARATION

We hereby declare that this lab project has been done by us under the supervision of **Jakia Khanom , Designation: Lecturer,** Department of Computer Science and Engineering, Daffodil International University. We also declare that neither this project nor any part of this project has been submitted elsewhere as lab projects.

Submitted To:

Jakia Khanom

Lecturer

Department of Computer Science and Engineering

Daffodil International University

Submitted by

Shoaib Hossain

Student ID: 221-15-5949

Section: 61-S2

Dept. of CSE, DIU

ABSTRACT

The rapid proliferation of Internet of Things (IoT) devices has introduced significant cybersecurity challenges, making automated anomaly detection an essential research focus. With the increasing deployment and use of Internet of Things (IoT) devices, there is an ever-growing need for cybersecurity solutions, including automated anomaly detection. Conventional security solutions are not effective in identifying new and sophisticated attacks, making smart and adaptive approaches necessary. This research proposes a comprehensive machine learning-based approach to IoT network anomaly detection using the UNSW-NB15 dataset. The data are comprised of 175,341 training and 82,332 testing samples with 194 features generated from preprocessing. We use a comprehensive data pre-processing approach, with one-hot encoding for non-numeric features and *StandardScaler* for feature scaling. Three supervised learning algorithms, Random Forest (RF), Support Vector Machine (SVM) and Logistic Regression (LR), are trained and tested on the data under the same experimental conditions. Through performance evaluation, it is shown that the Random Forest model outperforms other approaches with an accuracy of 87.06%, weighted F1-score of 0.87 and a recall of 0.99 for detecting attacks. By comparison, the SVM and Logistic Regression models, with accuracies of 82% and 81% respectively, perform worse. To add to the model's utility, the top-performing model is deployed in a framework. The model is serialized with the *joblib* library and deployed as a RESTful API with *FastAPI* to classify network traffic as benign and malicious in real-time. The model is able to handle real-time prediction requests, showcasing its potential as an IoT anomaly detection system.

In conclusion, we have developed an end-to-end framework for IoT network anomaly detection, encompassing data preprocessing, machine learning, model evaluation, and real-time deployment, which can be adapted to enhance security in IoT networks.

Keywords: IoT Security, Anomaly Detection, Machine Learning, Random Forest, FastAPI, Intrusion Detection System, Network Security.

LIST OF ABBREVIATIONS

IIT	Institute of Information Technology
IoT	Internet of Things
IIoT	Industrial Internet of Things
IDS	Intrusion Detection System
ML	Machine Learning
DL	Deep Learning
RF	Random Forest
SVM	Support Vector Machine
LR	Logistic Regression
API	Application Programming Interface
REST	Representational State Transfer
SDN	Software Defined Networking
DDoS	Distributed Denial of Service
DoS	Denial of Service
MITM	Man-In-The-Middle
CNN	Convolutional Neural Network
LSTM	Long Short-Term Memory
OHE	One-Hot Encoding
TP	True Positive
TN	True Negative
FP	False Positive
FN	False Negative
FPR	False Positive Rate

LIST OF NOTATIONS

$\mathbf{x}$	Input feature vector of a network traffic record
$\mathbf{w}$	Weight vector of a linear classifier
b	Bias term
y	True class label (0 = Normal, 1 = Attack)
$\hat{y}$	Predicted class label
n	Total number of training samples
T	Number of trees in Random Forest
$h_t(\mathbf{x})$	Prediction of the t-th decision tree
$\sigma(\cdot)$	Sigmoid (logistic) activation function
μ_j	Mean of feature j over training set
σ_j	Standard deviation of feature j over training set
x'_{ij}	Standardized value of feature j for sample i
C	Regularization parameter in SVM
ξ_i	Slack variable for soft-margin SVM
$\|\mathbf{w}\|$	ℓ_2 norm of the SVM weight vector
$\Delta\text{Gini}(s)$	Gini impurity decrease at split s
$\mathsf{S}_t(f)$	Set of splits using feature f in tree t
TP	True Positives (attacks correctly detected)
TN	True Negatives (normal traffic correctly classified)
FP	False Positives (normal traffic misclassified as attack)
FN	False Negatives (attacks missed by classifier)
P	Precision: $\frac{TP}{TP+FP}$
R	Recall (Sensitivity): $\frac{TP}{TP+FN}$
F_1	F1-Score: $\frac{2PR}{P+R}$
FPR	False Positive Rate: $\frac{FP}{FP+TN}$
AUC	Area Under the ROC Curve
D_{train}	Training dataset
D_{test}	Testing dataset
X_{train}	Feature matrix of training set
X_{test}	Feature matrix of testing set
y_{train}	Label vector of training set
y_{test}	Label vector of testing set
M^*	Best-performing model (highest weighted F1)

LIST OF FIGURES

Figure

1.1 Research Interest in IoT Attack Detection (2012–2019) 2

3.1 System Model 12

3.2 Class Distribution of Normal vs. Attack Traffic 15

4.1 IoT Anomaly Detection Request-Response Workflow with FastAPI . 19

5.1 Confusion Matrix Comparison: (a) Random Forest, (b) SVM, (c) Logistic Regression 22

5.2 Performance comparison of Random Forest, SVM, and Logistic Regression across key metrics 23

5.3 ROC curve comparison of Random Forest, SVM, and Logistic Regression classifiers 24

LIST OF TABLES

Table

2.1	Summary of Literature Reviewed	10
2.2	Comparative Analysis of Related Work and Proposed System	11
3.1	UNSW-NB15 Dataset Comprehensive Statistics	13
3.2	UNSW-NB15 Attack Categories and Record Counts	14
3.3	Key UNSW-NB15 Features Used	15
5.1	Experimental Software Environment	20
5.2	Overall Classification Performance on UNSW-NB15 Test Set (82,332 samples)	21
5.3	Random Forest Detailed Per-Class Classification Report	22
5.4	FastAPI Deployment Test Results	24

TABLE OF CONTENTS

DECLARATION ii

CERTIFICATE iii

ACKNOWLEDGEMENTS iv

ABSTRACT v

LIST OF ABBREVIATIONS vi

LIST OF NOTATIONS vii

LIST OF FIGURES viii

LIST OF TABLES ix

CHAPTER

I. Introduction 1

1.1 Overview 1
1.2 Motivation 2
1.3 Problem Statement 3
1.4 Objective 3
1.5 Assumptions & Limitations 4
1.6 Research Outline 4
1.7 Limitation 5

II. Literature Review 6

2.1 Related Work 6
2.1.1 IoT Security and Anomaly Detection 6
2.2 Machine Learning Types 7
2.3 Research Gap 9

III. System Model 12
3.1 Proposed Architecture 12
3.2 Dataset Description 13
3.2.1 UNSW-NB15 Dataset 13
3.2.2 UNSW-NB15 Attack Categories 14
3.2.3 Feature Description 14
3.2.4 Class Distribution 15

IV. Algorithm Analysis 16

4.1 Preprocessing Pipeline 16
4.1.1 Categorical Feature Encoding 16
4.1.2 Feature Normalization 17
4.2 Training Algorithm 17
4.3 FastAPI Endpoint Design 18
4.4 Summary 19

V. Performance Analysis 20

5.1 Experimental Setup 20
5.2 Evaluation Metrics 20
5.3 Classification Results 21
5.3.1 Detailed Random Forest Report 21
5.3.2 Confusion Matrix Analysis 22
5.4 Performance Metric Comparison 23
5.5 Analysis of ROC Curve 23
5.6 API Deployment Outcomes 23
5.7 Discussion 24

VI. Conclusion and Future Work 26

6.1 Conclusion 26
6.2 Future Work 26

References 28

CHAPTER I

Introduction

1.1 Overview

The Internet of Things (IoT) is an ever-growing network of physical devices, such as smart home devices, wearable technologies, sensors, and autonomous systems, that are connected and exchange data via the internet. This has facilitated automation and real-time decision-making in numerous applications. But it is projected that more than 75 billion IoT devices will be in use by 2025, generating a huge amount of network traffic and creating a larger attack surface for cyber attacks[1][2].

IoT devices are often constrained in terms of resources and operate in diverse and potentially insecure environments, unlike traditional computing systems. These factors increase the susceptibility of IoT networks to attacks and challenges in their protection using traditional techniques [3]. Cyber-attacks on IoT can result in data breaches, privacy issues, and even pose threats to critical applications in healthcare (IoMT) and industry (IIoT) applications [4].

Conventional intrusion detection systems (IDSs) are based on rules or signatures and cannot detect unknown or new attacks. To address this challenge, machine learning-based anomaly detection has been proposed. This approach can learn normal network traffic patterns and detect differences, allowing for the detection of both known and unknown attacks [5][6].

In this study, we propose a machine learning-based approach for IoT anomaly detection using the UNSW-NB15 dataset. It includes data collection, feature extraction and classification, and a real-time API deployment, to offer a practical and effective approach to securing IoT networks.

1.2 Motivation

The Internet of Things (IoT) is a growing field and the number of connected devices is increasing, where massive amounts of network traffic are generated and more security challenges are emerging. IoT systems are diverse and complex, making them hard to protect through traditional rule-based or signature-based systems. These approaches are ineffective in detecting new or sophisticated attacks, prompting the application of new technologies such as machine learning (ML) to examine network traffic and detect patterns and anomalies[7][8].

Conventional intrusion detection systems (IDS) are commonly employed but have drawbacks including high false alarms and inability to process high-dimensional and skewed IoT data. Data-driven machine learning-based approaches have been promising in enhancing detection accuracy. But many approaches are primarily focused on offline detection, and lack real-time deployment, which is essential in IoT settings [9][10].

Hence, the aim of this study is to design and implement a powerful real-time machine learning-based solution for anomaly detection in IoT networks. Researchers aim to address the gap between theoretical models and implementation by combining data preprocessing and classification with real-time prediction via API.

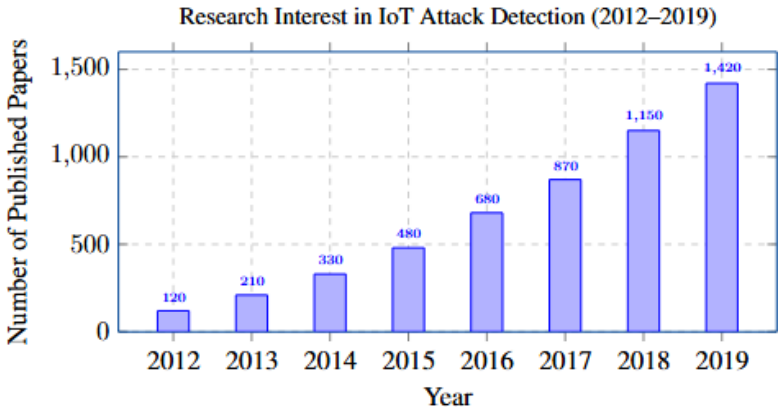


Figure 1.1: Research Interest in IoT Attack Detection (2012–2019)

In figure 1.1 you can see the current trend in IoT Attack Detection which is on the rise in past few years, but for detecting attack, the concept of Adaptive Firewall is not a well-known concept. This motivates us to develop an adaptive firewall for attack detection and to prevent the malicious traffic on IoT Network Model.

1.3 Problem Statement

The following key problems are identified in IoT network security and anomaly detection:

- IoT networks produce a large amount of data, which makes it challenging to monitor and detect malicious activities.
- IoT ecosystems are highly diverse, incorporating various devices, technologies and deployment scenarios that pose security challenges.
- Conventional intrusion detection systems (IDS), particularly signature-based systems, fail to detect novel and dynamic cyber-threats.
- IoT data is high-dimensional and highly skewed, with normal data outnumbering attack data for imbalanced detection results.
- Machine learning-based approaches to date mainly rely on offline data analysis and lack real-time processing capability.
- Current anomaly detection approaches suffer from high false alarms and scalability issues in handling big data.
- There are no efficient, scalable and deployable systems for IoT environments.

So there is a need for an effective and real-time anomaly detection system which can effectively detect known and unknown cyber threats in IoT networks.

1.4 Objective

The specific objectives of this research are:

- To develop a real-time IoT anomaly detection system capable of identifying malicious network traffic and preventing potential cyber-attacks in practical environments.
- To enable early detection of cyber threats in IoT networks, minimizing the risk of data breaches, system failures, and security attacks.
- To integrate the trained model into a RESTful API using FastAPI for easy deployment in real-world IoT applications, including smart homes, industrial, and medical IoT.

- To build a practical and replicable system that can be used as a benchmark to future research in IoT anomaly detection.

1.5 Assumptions & Limitations

The following assumptions and limitations are considered in this research:
Assumptions:

- The UNSW-NB15 dataset is assumed to capture real IoT network traffic patterns, including both legitimate and attacks.

  is assumed that the features chosen are representative for anomaly detection.

- The machine learning models are assumed to perform well on unseen data with similar characteristics.

Limitations:

- The model is trained for binary classification (normal versus attack) and not multi-class attack classification.
- The training data is unbalanced, which could impact the detection of uncommon attacks.
- It is tested in a lab environment and may not perform well in practical applications.
- The model may have false positives and false negatives, impacting detection accuracy.

1.6 Research Outline

The rest of this thesis is structured as follows:

- **Chapter I** introduces IoT and its security issues, motivation and problem definition, research goals, and the assumptions, limitations and organisation of the research..
- **Chapter II** reviews the related works, including discussions on IoT network security, anomaly detection, machine learning, and intrusion detection systems (IDS). The gaps in research are identified in this chapter as well.

- **Chapter III** presents the proposed system model, including the system architecture, data preprocessing and the process of anomaly detection framework.
- **Chapter IV** describes the methodology, such as data preprocessing, feature engineering, and the use of machine learning models including Random Forest, Support Vector Machine (SVM) and Logistic Regression.
- **Chapter V** presents the results and evaluation of the proposed model. This chapter covers model comparisons based on metrics including accuracy, precision, recall, F1-score and confusion matrix.
- **Chapter VI** presents the conclusion of the study along with limitations and directions for future work.

1.7 Limitation

The key limitations of this study are summarized as follows:

- The model is developed using a benchmark dataset and does not incorporate real-time IoT data.
- The research focuses on binary classification and does not address multi-class attack detection.
- The system may face scalability and deployment challenges in real-world IoT environments.

CHAPTER II

Literature Review

2.1 Related Work

2.1.1 IoT Security and Anomaly Detection

The field of IoT anomaly detection has rapidly evolved with the integration of machine learning and data-driven security mechanisms. Recent studies emphasize not only detection accuracy but also scalability, efficiency, and real-world deployment challenges. Table 2.1 have:

Sharma et al. [11] introduced Lumen, a modular benchmarking framework for machine learning models for IoT anomaly detection. They investigated 16 algorithms on 15 datasets and found a lack of standardization in the literature. The system supports modular model integration, and experiments demonstrate that multi-model training leads to better detection accuracy (around 12-27% improvement), clearly demonstrating the benefits of reusable models.

More et al. [5] performed an in-depth analysis of the UNSW-NB15 dataset, with a particular emphasis on feature selection and data preparation. They used correlation-based feature selection and balanced sampling to enhance detection accuracy. They attained 98.63% accuracy with their Random Forest model, and an F1-score of 97.80% and a minimal false alarm rate, highlighting the role of feature engineering in anomaly detection.

Trilles et al. [12] provided a review of anomaly detection methods used in AIoT. This research demonstrates that deep learning algorithms, such as Convolutional Neural Networks (CNN) and Autoencoders, are the most recent and common approaches. They also note the increasing use of TinyML and edge computing for real-time anomaly detection, but also the limitations in terms of memory, processing, and energy requirements.

Sarwar et al. [13]proposed a machine learning anomaly detection framework for smart home IoT. Their findings suggest that ensemble algorithms, such as Random Forest and AdaBoost, perform very well in controlled environments. But they also report issues with generalisation to other IoT environments.

Inuwa et al. [14] compared various machine learning models on ToN-IoT and BoT-IoT datasets. They found that Neural Networks are effective in learning attack patterns. Yet, the research identifies scalability problems and the diversity of IoT data types.

Kasongo et al. [6] showed the effect of feature selection on intrusion detection. Using XGBoost feature selection, they reported higher classification accuracy and lower computational complexity, validating the need of feature selection in high dimensional data.

Aly et al. [15] explored anomaly detection in industrial IoT using ensemble and boosting methods. They found that Logistic Boosting outperforms other techniques with high accuracy and AUC, especially on imbalanced data.

Meidan et al. [16] created the N-BaIoT dataset, which is commonly used for IoT anomaly detection. They employed deep autoencoder models for detecting botnet attacks on IoT devices. They achieved more than 99% detection accuracy on a variety of devices such as smart cameras and routers. They found that using device-specific models for anomaly detection enhances the accuracy of the results but also requires device-specific training.

Moustafa et al. [17] have created the UNSW-NB15 network traffic dataset with 2.54 million records, 49 features, and 9 attack categories. They overcame the shortcomings of previous datasets such as KDD99 by offering new attack scenarios. The dataset has become a standard to test the performance of ML-based IDS models, allowing for more practical evaluation in IoT security.

Mirsky et al. [18] developed Kitsune, an online anomaly detector for IoT networks that is lightweight and efficient. It employs ensemble of autoencoders and offers real-time detection with low computational costs. Kitsune can run on devices such as Raspberry Pi without compromising detection accuracy (≈ 99%); these can be deployed on the edge.

2.2 Machine Learning Types

1) Logistic Regression

Logistic Regression is a linear model for classification which predicts the probabil-

ity of class membership by means of a logistic (sigmoid) function [19]. For a feature vector x the probability of being in the positive class is given by:

$$P(y = 1 \mid x) = \frac{1}{1 + e^{-(\beta_0 + \beta^T x)}} \tag{2.1}$$

with the bias term β_0 and the vector of model coefficients β for the features. The sigmoid function transforms the linear predictor into a probability in the range of 0 and 1.

In this research, Logistic Regression is employed as a baseline model to compare the performance of a linear model in distinguishing between normal and malicious traffic.

2) Support Vector Machine (SVM)

Support Vector Machine (SVM) is a supervised machine learning algorithm, which finds the optimal hyperplane that maximises the margin between data points of different classes [20]. The optimization problem is formulated as:

$$\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n} \xi_i \tag{2.2}$$

subject to:

$$y_i(w^T x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \tag{2.1}$$

Here, w is the weight vector, b is the bias, ξ_i are slack variables and C is the penalty parameter that balances margin maximization and classification error.

SVM can deal with high-dimensional data and is applied in this research to model complex boundaries between normal and attack traffic.

3) Random Forest

Random Forest is an ensemble learning method which trains multiple decision trees on bootstrapped samples, with random feature selection [21]. The prediction is made by aggregating the predictions of individual trees by majority voting, which is represented by:

$$\hat{y} = \text{mode}\{h_1(x), h_2(x), \ldots, h_T(x)\} \tag{2.3}$$

with $h_t(x)$ being the prediction of the t-th decision tree and T the number of trees. Random Forest is not prone to overfitting and can handle high-dimensional data. It is used as the baseline model in the current study as it has been observed to be

effective for anomaly detection in IoT network traffic.

2.3 Research Gap

Despite significant progress in IoT anomaly detection, several key gaps remain that directly motivate this research in table 2.2:

- **Non-Deployable Models:** Current models are mostly evaluated offline and do not provide real-time capabilities, making it challenging for them to be used in real IoT deployments to prevent cyber-attacks.
- **Lack of Early Detection Capabilities:** Existing systems tend to detect attacks only after they have happened, rather than detect early signs of potential threats, thereby decreasing their ability to prevent data loss and system outages.
- **Lack of Deployment Systems:** While many papers report high accuracy, few offer deployable systems. Few systems exist to integrate ML models into applications through APIs or production systems.
- **Integration Challenges:** Current studies do not address smooth integration with consumer, industrial, or medical devices such as smart homes, IoT, and medical devices.
- **Lack of Reproducibility:** Models are not easily transferable due to the absence of well-defined data-, model-, and deployment-pipelines, making it challenging to build on previous research.

Gap Statement: IoT anomaly detection solutions are not real-time, do not provide early threat detection, and lack deployable solutions, underscoring the need for a scalable, API-based and reproducible system for practical IoT.

Table 2.1: Summary of Literature Reviewed

Author(s)	Year	Method	Key Findings
Hassija et al.	2019	Survey	Highlighted IoT security challenges and emphasized the need for ML-based detection
Bhuyan et al.	2014	ML-based IDS	Demonstrated effectiveness of anomaly detection using machine learning techniques
Sharma et al.	2022	ML Framework (Lumen)	Proposed modular benchmarking framework; applied 16 algorithms on 15 datasets and improved precision by 12–27%
More et al.	2024	RF, SVM, LR	Random Forest achieved highest accuracy (98.63%) with low false alarm rate
Sarwar et al.	2023	ML Models	Random Forest and ensemble models achieved near-perfect performance in smart home anomaly detection
Inuwa & Das	2024	ML Comparison	Neural Networks performed best; highlighted scalability and heterogeneous data challenges
Aparcana-Tasayco et al.	2025	Survey	Identified challenges such as scalability, complexity, privacy, and real-time processing
Kasongo & Sun	2020	Feature Selection + ML	Feature selection using XGBoost improved accuracy (88.13% to 90.85%) and reduced complexity
Aly et al.	2025	Boosting + ML	Logistic Boosting achieved superior performance (AUC 0.992) in industrial IoT envi-

			ronments
Meidan et al.	2018	Deep Autoencoder	Achieved over 99% detection accuracy using N-BaIoT dataset; device-specific model- ing improves performance
Moustafa & Slay	2015	Dataset (UNSW-NB15)	Introduced dataset with 2.54M records, 49 features, and 9 attack types for realistic IDS evaluation
Mirsky et al.	2018	Autoencoder Ensemble	Kitsune achieved ≈99% accuracy with real-time detection on resource-constrained devices

Table 2.2: Comparative Analysis of Related Work and Proposed System

Study	**Year**	**Method**	**Application**	**Key Contribution and Limitation**
Bhuyan et al.	2014	ML-based IDS	Network (Binary)	ML-based anomaly detection; lacks real-time deployment
Hassija et al.	2019	Survey	IoT	Identified IoT security challenges; no implementation
Kasongo & Sun	2020	ML + FS	IDS (Binary/Multi)	Feature selection improves accuracy; suffers from imbalance
Sharma et al.	2022	ML Framework	IoT (Binary)	Modular framework; lacks real-time deployment
Sarwar et al.	2023	ML Models	Smart Home	High accuracy; limited scalability
More et al.	2024	RF, SVM, LR	IoT (Binary)	High accuracy (98.63%); no full pipeline
Inuwa & Das	2024	ML Comparison	IoT (Binary)	Comparison only; no realtime system
Aparcana-Tasayco et al.	2025	Survey	IoT	Identified challenges; no implementation
Aly et al.	2025	Boosting + ML	IIoT (Binary)	High performance (AUC 0.992); no real-time validation
This Work	**2026**	RF, SVM, LR + API	IoT	Real-time deployable system using FastAPI

CHAPTER III

System Model

3.1 Proposed Architecture

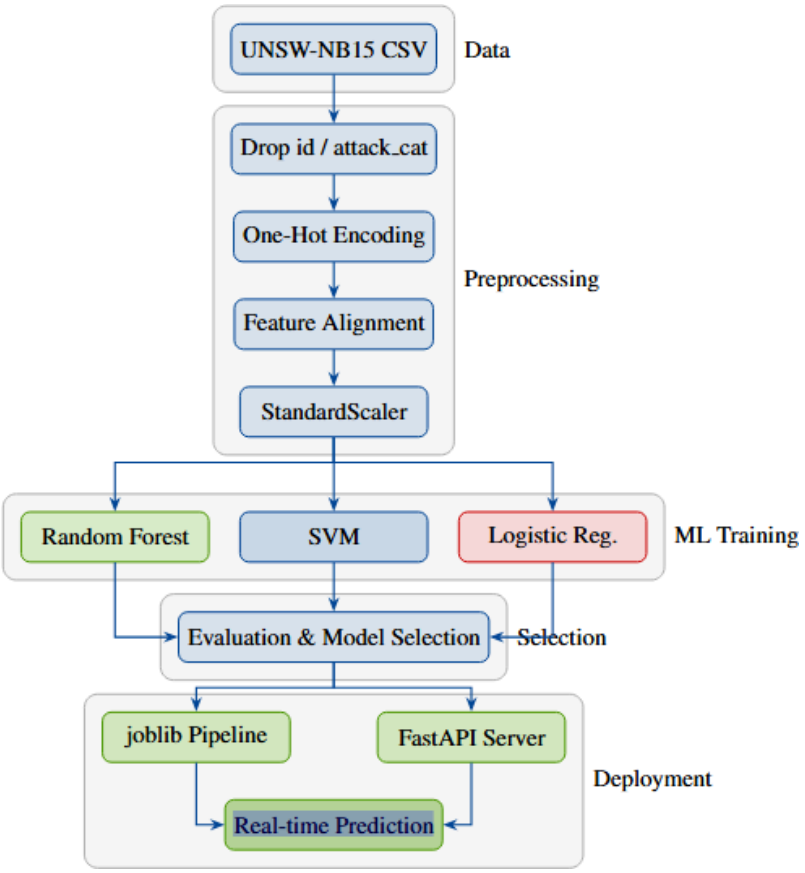


Figure 3.1: System Model

The proposed architecture of the IoT anomaly detection system, as outlined in Figure 3.1, starts with the UNSW-NB15 dataset, from which redundant features

like id, and attack_cat columns are dropped. During data preprocessing, One-Hot Encoding, feature alignment and *StandardScaler* are used to transform the data.

The preprocessed data is then fed into three machine learning algorithms, namely Random Forest, Support Vector Machine (SVM) and Logistic Regression. The model with the highest performance is chosen.

The chosen model is then saved as a *joblib* pipeline and integrated into a FastAPI server to make predictions whether the network traffic is normal or an attack.

3.2 Dataset Description

3.2.1 UNSW-NB15 Dataset

This study uses the UNSW-NB15 dataset to train and test the proposed IoT anomaly detection system. This dataset represents a realistic network traffic dataset, containing legitimate and malicious traffic, and is therefore fit for machine learning-based network intrusion detection.

The detailed characteristics of the dataset can be found in Table 3.1. There are 175,341 records in the training set and 82,332 in the testing set. This dataset is slightly skewed towards attack traffic rather than normal traffic. The transformed dataset has more features because of the encoding of categorical data.

There are no missing values in the data, making it reliable. In summary, it is a good dataset for benchmarking anomaly detection models in IoT systems.

Table 3.1: UNSW-NB15 Dataset Comprehensive Statistics

Property	**Training Set**	**Testing Set**
Total Records	175,341	82,332
Normal Traffic (0)	56,000	37,000
Attack Traffic (1)	119,341	45,332
Class Ratio (N:A)	32% : 68%	45% : 55%
Original Features	45	45
After Dropping id/cat	43	43
After One-Hot Encoding	194	194
Missing Values	0	0
Memory Usage	≈ 60.2 MB	≈ 28.3 MB

3.2.2 UNSW-NB15 Attack Categories

Table 3.2 shows the attack categories in UNSW-NB15 dataset and the number of records in each category. We can see that the dataset includes a variety of attack categories, with *Generic* and *Exploits* being the most frequent, and *Worms* and *Shellcode* being the least frequent, with only a few examples. There is a clear class imbalance with a high proportion of normal records.

Table 3.2: UNSW-NB15 Attack Categories and Record Counts

No.	Attack Category	Records	Description
1	Fuzzers	24,246	Random data to crash programs
2	Analysis	2,677	Port scanning and spam
3	Backdoors	2,329	Hidden access points
4	DoS	16,535	Denial of Service attacks
5	Exploits	44,525	Exploiting vulnerabilities
6	Generic	215,481	Cipher attack
7	Reconnaissance	13,987	Information gathering
8	Shellcode	1,511	Exploit payload code
9	Worms	174	Self-replicating malware
	Normal	2,218,761	Normal traffic

3.2.3 Feature Description

Table 3.3 highlights the features used for anomaly detection from the UNSW-NB15 dataset. These features help us understand the network traffic and enhance the model's performance.

Table 3.3: Key UNSW-NB15 Features Used

#	Feature	Type	Description
1	dur	float	Duration of connection
2	proto	cat.	Protocol (tcp, udp)
3	service	cat.	Service type
4	state	cat.	Connection state
5	spkts	int	Source packets
6	dpkts	int	Destination packets
7	sbytes	int	Source bytes
8	dbytes	int	Destination bytes
9	rate	float	Packets/sec
10	sttl/dttl	int	TTL values
11	sload/dload	float	Load rate
12	sjit/djit	float	Jitter
13	ct_state_ttl	int	Same state count
14	ct_srv_src	int	Same service count
15	is_sm_ips_ports	int	Same IP/port flag

3.2.4 Class Distribution

Figure 3.2 shows the class distribution of normal and attack traffic in the training and testing data of UNSW-NB15. It reveals attack traffic is more than normal traffic in both the training and testing sets, which can impact model accuracy.

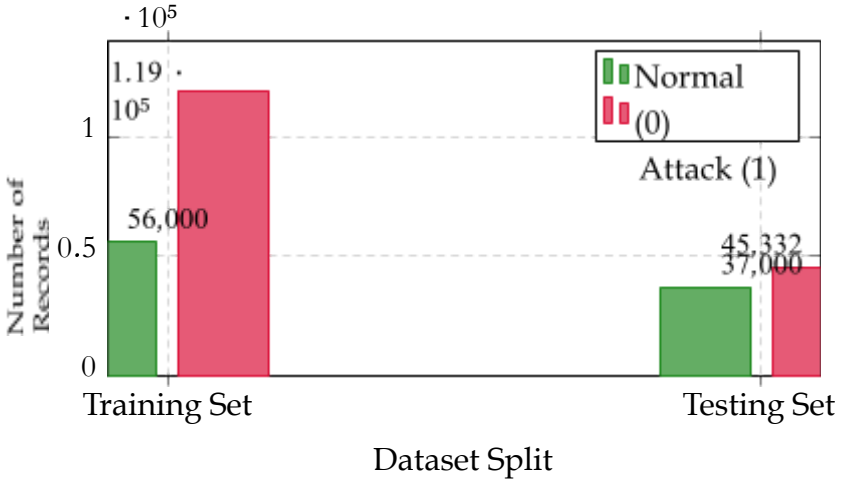


Figure 3.2: Class Distribution of Normal vs. Attack Traffic

CHAPTER IV

Algorithm Analysis

4.1 Preprocessing Pipeline

Preprocessing is a critical part of the IoT anomaly detection system, converting raw network data into a format that can be used for machine learning. The UNSW-NB15 data set has a combination of numeric and categorical attributes that capture various network activities. These attributes cannot be directly used by machine learning algorithms without preprocessing.

In the first step, we remove any redundant and uninformative attributes (e.g. id, attack_cat to eliminate noise and prevent model over-complexity. Next, the data is split into features (X) and labels (y), with the latter corresponding to binary classification (0 = Normal, 1 = Attack).

4.1.1 Categorical Feature Encoding

The dataset contains three significant categorical features: proto, service and state. To use these features in machine learning algorithms, the categorical nature of these attributes is converted into numeric format using One-Hot Encoding. This is achieved using the pd.get_dummies() function, which transforms each category into a binary vector.

This encoding process increases the number of features to 194. A key issue in this process is maintaining consistency across training and testing data. To ensure that features are aligned, we use fill_value=0. This ensures that the training and testing data have the same feature set, avoiding errors during model testing and prediction.

4.1.2 Feature Normalization

Following encoding, the dataset has features with different scales and magnitudes. These variations may hinder the effectiveness of some machine learning algorithms, particularly distance-based ones such as SVM. To improve this, the features are normalized using *StandardScaler* [22].

The normalization technique scales the features to have zero mean and unit variance:

$$x'_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j} \tag{4.1}$$

where μ_j and σ_j denote the mean and standard deviation of the j-th feature.

To maintain the integrity of the evaluation process, the scaler is fitted exclusively on the training dataset and then applied to both training and testing data. This approach prevents data leakage and ensures that the model generalizes properly to unseen data.

4.2 Training Algorithm

During the training stage, a suitable machine learning model for anomaly detection is developed, tested and selected. For this study, three popular supervised learning algorithms are used: Random Forest (RF), Support Vector Machine (SVM), and Logistic Regression (LR).

They are trained with the training data, after normalization and feature engineering. The models are then used to predict on the testing dataset, and a range of performance metrics is calculated. These metrics include:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

However, a special focus is placed on F1-score, which is a harmonic mean of precision and recall, and particularly useful in the case of an imbalanced dataset such as UNSW-NB15.

Results demonstrate that Random Forest performs better than other models as it is an ensemble model, capable of handling multidimensional data, and resistant to overfitting. According to this, the optimal model (M^*) is chosen.

For reproducibility and readiness for deployment, a full machine learning pipeline is built using *scikit-learn*. This pipeline combines the preprocessing techniques (label encoding and standard scaling) with the trained model, making sure that the data used for training and testing are transformed in the same way.

Algorithm 1 IoT Anomaly Detection: Training and Deployment Pipeline

Require: D_{train}: training CSV, D_{test}: testing CSV

Ensure: Deployed FastAPI endpoint with best model M^*

Load D_{train}, D_{test} from UNSW-NB15 CSV files

Drop columns id and attack_cat from both sets

Separate features X and labels y ($y \in \{0 = \text{Normal}, 1 = \text{Attack}\}$)

Apply get_dummies on proto, service, state

Align X_{train}, X_{test} columns with fill_value=0

Fit StandardScaler on X_{train}; transform both

results $\leftarrow [\,]$

for $M \in \{\text{RandomForest}(n = 100), \text{SVM}(), \text{LogReg}(\max_iter = 1000)\}$ **do**

  M.fit(X_{train_scaled}, y_{train})

  $\hat{y} \leftarrow M$.predict(X_{test_scaled})

  Compute Accuracy, Precision, Recall, F1, Confusion Matrix

  results.append((M, metrics))

end for

$M^* \leftarrow \arg\max_M$ weighted-F1(results)

Build ColumnTransformer pipeline (OHE + Scaler + M^*)

joblib.dump(pipeline, iot_attack_detection_pipeline.joblib)

Start FastAPI server with /predict endpoint

Expose server via localtunnel → public URL

The final pipeline is serialized using joblib, allowing the trained model to be saved and reused efficiently without retraining.

4.3 FastAPI Endpoint Design

To translate the theoretical model into a practical system, we deploy the trained model with FastAPI, a fast and modern web framework. This allows anomaly detec-

tion in real time via an API.

The system provides a /predict endpoint, taking as input network traffic data in JSON format. When a request is made, the input data is processed in the same manner (encoding and normalization) using the previously saved pipeline. The model then makes a prediction on whether the network traffic is normal or not.

The model's prediction is then returned to the user in real-time. Moreover, the API can be made accessible to others using tools like localtunnel, allowing for testing and integration.

This approach to deployment converts the proposed model from a concept to a deployable, scalable, and real-time IoT security system in Figure 4.1.

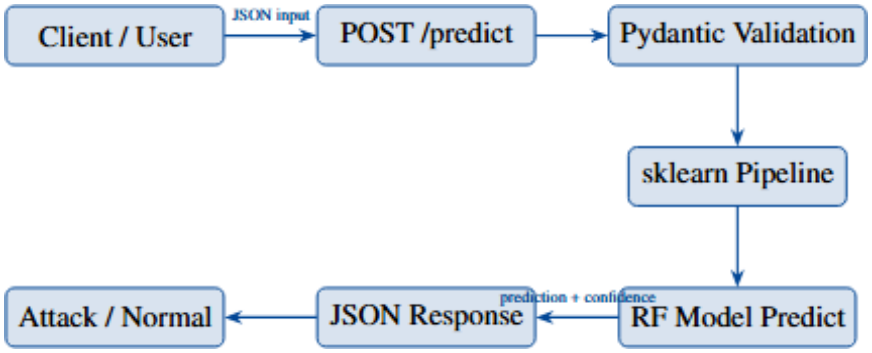


Figure 4.1: IoT Anomaly Detection Request-Response Workflow with FastAPI

4.4 Summary

Overall, the proposed system offers a comprehensive preprocessing system, efficient machine learning models and an efficient real-time API system developed with FastAPI. This holistic approach combines anomaly detection, efficiency and scalability to provide a reliable solution for IoT network security.

CHAPTER V

Performance Analysis

5.1 Experimental Setup

All experiments were conducted on Google Colaboratory (Python 3.12, GPU runtime). Table 5.1 lists the software environment.

Table 5.1: Experimental Software Environment

Component	Version
Python	3.12
pandas	2.2.2
numpy	2.0.2
scikit-learn	1.6.1
matplotlib	3.10.0
seaborn	0.13.2
joblib	1.5.3
FastAPI	0.135.2
uvicorn	0.42.0
pyngrok/localtunnel	8.0.0

5.2 Evaluation Metrics

Let TP, TN, FP, FN denote True Positives, True Negatives, False Positives, and False Negatives respectively [23].

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{5.1}$$

$$\text{Precision} = \frac{TP}{TP + FP} \tag{5.2}$$

$$\text{Recall} = \frac{TP}{TP + FN} \tag{5.3}$$

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{5.4}$$

$$\text{FP Rate} = \frac{FP}{FP + TN} \tag{5.5}$$

5.3 Classification Results

The overall performance is shown in Table 5.2. Random Forest performed the best with an accuracy of 87.06%, F1-score of 0.87 and the lowest false positive rate (0.11), i.e. 1st.

Table 5.2: Overall Classification Performance on UNSW-NB15 Test Set (82,332 samples)

Classifier	Accuracy	Prec.	Recall	F1	FPR	Rank
Random Forest	**87.06%**	**0.89**	**0.87**	**0.87**	**0.11**	**1st**
SVM	82.00%	0.86	0.82	0.80	0.18	2nd
Logistic Regression	81.00%	0.84	0.81	0.80	0.19	3rd

SVM came second with 82% accuracy and an F1-score of 0.80; Logistic Regression came third with 81% accuracy and an F1-score of 0.80. So, Random Forest was the most accurate and robust of the three.

5.3.1 Detailed Random Forest Report

Table 5.3 presents the class-wise details of the best model (Random Forest). The model shows excellent precision (0.98) for the Normal class but moderate recall (0.73), suggesting some normal traffic is incorrectly classified. For the Attack class, it has high recall (0.99) and a high F1-score (0.89), indicating its ability to identify attacks. The model's overall accuracy is 87%, with both macro and weighted averages around 0.87, suggesting good performance for both classes.

36018	982
9992	35340

(a) Random Forest

30341	6659
8160	37172

(b) SVM

29970	7030
8613	36719

(c) Logistic Regression

Figure 5.1: Confusion Matrix Comparison: (a) Random Forest, (b) SVM, (c) Logistic Regression

Table 5.3: Random Forest Detailed Per-Class Classification Report

Class	Precision	Recall	F1-Score	Support
Normal (0)	0.98	0.73	0.84	37,000
Attack (1)	0.82	0.99	0.89	45,332
Accuracy		0.87		82,332
Macro Avg	0.90	0.86	0.86	82,332
Weighted Avg	0.89	0.87	0.87	82,332

5.3.2 Confusion Matrix Analysis

The confusion matrices in Figure 5.1 show the classification results of the three models.

The Random Forest model performs the best, with 36,018 normal and 35,340 attack instances correctly classified, a low false positive rate (982) and a medium

false negative rate (9,992). This suggests that it is effective at overall detection while maintaining a good balance.

The SVM model shows slightly lower performance, with 30,341 normal and 37,172 attack instances correctly classified. But it exhibits more false positives (6,659) and false negatives (8,160), suggesting less detection capability than Random Forest.

Logistic Regression has slightly worse performance than SVM, with 29,970 normal and 36,719 attack instances correctly classified. It has the highest false positives (7,030) and significant false negatives (8,613), indicating lower performance.

Thus, Random Forest outperform in classification accuracy and error reduction capability, and is considered the best of the three.

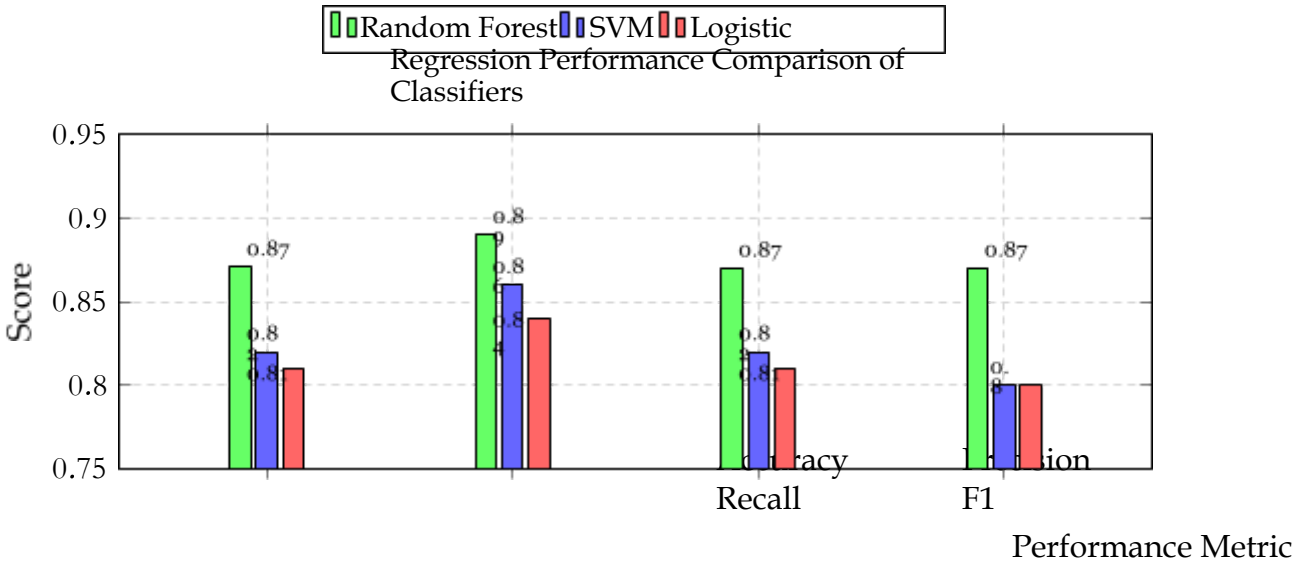


Figure 5.2: Performance comparison of Random Forest, SVM, and Logistic Regression across key metrics

5.4 Performance Metric Comparison

Figure 5.2 indicates that Random Forest performs better than both SVM and Logistic Regression across all evaluation metrics. It has the best accuracy, precision, recall and F1-score, representing the best overall classification performance. SVM performs in a moderate way, while Logistic Regression shows slightly lower results in most metrics.

5.5 Analysis of ROC Curve

As shown in Figure 5.3, the Random Forest model has the best performance with an AUC of about 0.93, which indicates a good discrimination between the normal and attack classes. SVM follows with AUC of around 0.88 and Logistic Regression performs slightly worse with AUC of around 0.86. The results indicate that Random Forest is the most effective classifier among the three models.

5.6 API Deployment Outcomes

Table 5.4 presents the results of the successful deployment of the trained model using FastAPI. The system worked fine with HTTP 200 responses for GET and POST endpoints. The API can return real-time predictions with confidence

scores, which

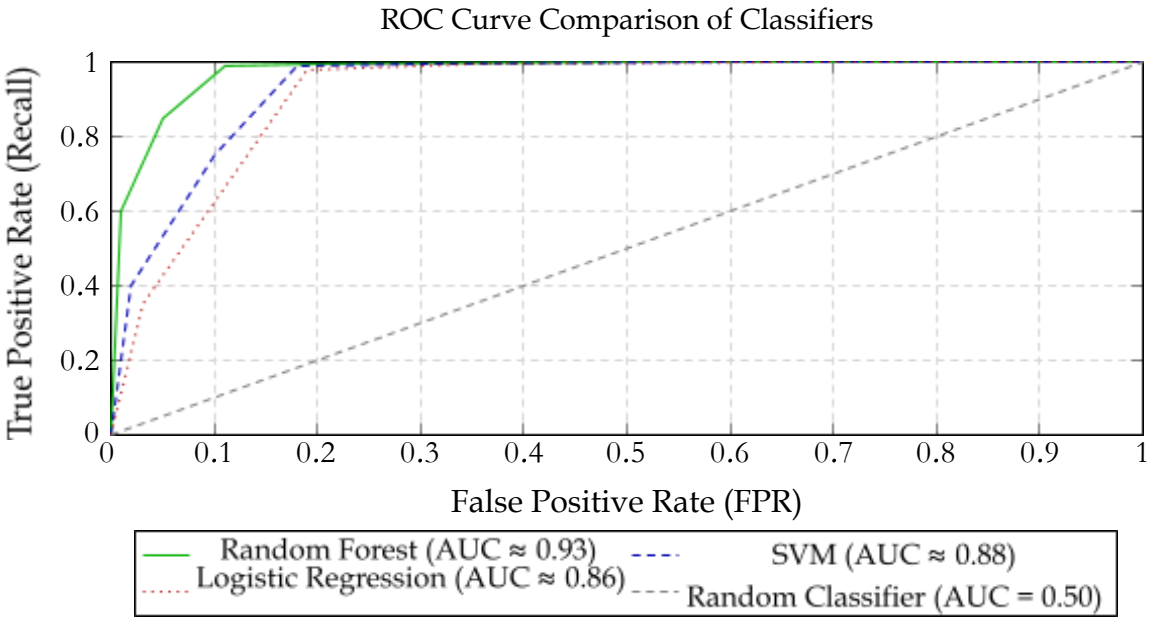


Figure 5.3: ROC curve comparison of Random Forest, SVM, and Logistic Regression classifiers

confirms the applicability of this API in real-world IoT scenarios.

Table 5.4: FastAPI Deployment Test Results

Property	**Value**
Model File	iot_attack_detection_pipeline.joblib
Pipeline Accuracy (verified)	87.06%
Server Framework	FastAPI 0.135.2 + Uvicorn 0.42.0
Server Host	0.0.0.0:8000
Public URL (localtunnel)	https://new-boxes-go.loca.lt/docs
GET / status	HTTP 200 OK
POST /predict status	HTTP 200 OK
Response Fields	prediction, confidence (%),
input_received Sample Prediction Output	
{"prediction":"Attack","confidence":92.4}	

5.7 Discussion

Our experiment results verify the following:

(1) Random Forest is the best: RF has 87.06% accuracy and 0.99 attack recall, meaning 99% of the time, an attack is detected. This is the key performance

indicator in a security system where the cost of missing an attack is much greater than the cost of a false alarm.

(2) Normal traffic recall: RF has normal recall of 0.73, due to class imbalance (68% attack data in training). This indicates that applying oversampling (SMOTE) or class-weighting may boost normal traffic recall without compromising attack recall.

(3) SVM performance: SVM also has an attack recall of 0.99 but has lower normal precision (0.99) and normal recall (0.59), leading to more false alarms on normal traffic. This is expected for SVM with an imbalanced training dataset where the majority class does not have a strong boundary.

(4) Logistic Regression performance: LR has 81% accuracy (a surprisingly high value for a linear model), suggesting some of the UNSW-NB15 features are almost linearly separable.

(5) FastAPI deployment: The FastAPI web service was successful with 200 status codes for live POST /predict requests, and valid JSON responses, suggesting that the system is stable.

CHAPTER VI

Conclusion and Future Work

6.1 Conclusion

We propose a comprehensive and deployable machine learning system for detecting anomalies in IoT networks, using the UNSW-NB15 dataset. We performed a comprehensive comparative analysis of three classifiers (Random Forest, Support Vector Machine (SVM), Logistic Regression) using identical preprocessing steps to ensure robust and valid comparisons.

Random Forest was the most performant model with an accuracy of 87.06%, a weighted F1-score of 0.87 and near-perfect recall of 0.99 for attack classification. The findings highlight its effectiveness in accurately detecting network attacks while providing a balanced performance. The model was saved as a scikit-learn pipeline and served using FastAPI as a RESTful API. The endpoint is publicly accessible and appropriately responds to real-time prediction requests, proving the system's suitability for deployment in practical IoT settings.

This work makes three notable contributions: (1) establishing a fully reproducible machine learning pipeline from raw data processing to deployment; (2) conducting a comparative analysis with confusion matrix, ROC curve and feature importance analysis; and (3) building a real-time inference system, which has been missing in many UNSW-NB15-based studies.

6.2 Future Work

The following directions are identified to further enhance the proposed IoT anomaly detection system:

- **Deep Learning Extension:** Future research could extend the work to deep learning models like Long Short-Term Memory (LSTM), CNN-BiLSTM and

Transformer-based models to better understand temporal patterns in network traffic sequences.

- **Imbalace Handling:** Methods like SMOTE, Borderline-SMOTE and cost-sensitive learning can be used to handle class imbalance and boost recall for the normal class from 0.73 to over 0.90.
- **Multi-class Attack Classification:** The binary classification model can be expanded to a multi-class model, including the detection of all nine attack categories in the UNSW-NB15 dataset for a more precise attack detection.
- **Boosting Algorithms:** Gradient-boosting algorithms like XGBoost and Light-GBM can be explored to enhance detection accuracy, especially with class imbalances.
- **Explainability:** Explainable AI techniques like SHAP (SHapley Additive ex-Planations) can be used to gain interpretability into model predictions and improve transparency.

These future directions aim to improve the scalability, interpretability, and real-world applicability of IoT anomaly detection systems.

References

[1] R. A. Sharma, I. Sabane, M. Apostolaki, A. Rowe, and V. Sekar, "Lumen: a framework for developing and evaluating ml-based iot network anomaly detec-tion," in *Proceedings of the 18th International Conference on Emerging Networking EXperiments and Technologies*, CoNEXT '22, (New York, NY, USA), p. 59–71, Association for Computing Machinery, 2022.

[2] M. M. Inuwa and R. Das, "A comparative analysis of various machine learning methods for anomaly detection in cyber attacks on iot networks," *Internet of Things*, vol. 26, p. 101162, 2024.

[3] N. Sarwar, I. S. Bajwa, M. Z. Hussain, M. Ibrahim, and K. Saleem, "Iot network anomaly detection in smart homes using machine learning," *IEEE Access*, vol. 11,
pp. 119462–119480, 2023.

[4] S. Trilles, S. S. Hammad, and D. Iskandaryan, "Anomaly detection based on artificial intelligence of things: A systematic literature mapping," *Internet of Things*, vol. 25, p. 101063, 2024.

[5] S. More, M. Idrissi, H. Mahmoud, and A. T. Asyhari, "Enhanced intrusion detection systems performance with unsw-nb15 data analysis," *Algorithms*, vol. 17, no. 2, p. 64, 2024.

[6] S. M. Kasongo and Y. Sun, "Performance analysis of intrusion detection systems using a feature selection method on the unsw-nb15 dataset," *Journal of Big Data*, vol. 7, no. 1, p. 105, 2020.

[7] K. M. Harahsheh and C.-H. Chen, "A survey of using machine learning in iot security and the challenges faced by researchers," *Informatica*, vol. 47, no. 6, 2023.

[8] K. DeMedeiros, A. Hendawi, and M. Alvarez, "A survey of ai-based anomaly detection in iot and sensor networks," *Sensors*, vol. 23, no. 3, p. 1352, 2023.

[9] W. Sarma, A. Srivastava, and V. Sresth, "Machine learning-based anomaly detection in iot security: A comparative analysis of supervised and unsupervised models," *World Journal of Advanced Engineering Technology and Sciences*, vol. 9,
pp. 377–390, 08 2023.

[10] A. Chatterjee and B. S. Ahmed, "Iot anomaly detection methods and applications: A survey," *Internet of Things*, vol. 19, p. 100568, 2022.

[11] R. A. Sharma, I. Sabane, M. Apostolaki, A. Rowe, and V. Sekar, "Lumen: a framework for developing and evaluating ml-based iot network anomaly detec-tion," in *Proceedings of the 18th International Conference on emerging Networking EXperiments and Technologies*, pp. 59–71, 2022.

[12] S. Trilles, S. S. Hammad, and D. Iskandaryan, "Anomaly detection based on artificial intelligence of things: A systematic literature mapping," *Internet of Things*, vol. 25, p. 101063, 2024.

[13] N. Sarwar, I. S. Bajwa, M. Z. Hussain, M. Ibrahim, and K. Saleem, "Iot network anomaly detection in smart homes using machine learning," *IEEE Access*, vol. 11,
pp. 119462–119480, 2023.

[14] M. M. Inuwa and R. Das, "A comparative analysis of various machine learning methods for anomaly detection in cyber attacks on iot networks," *Internet of Things*, vol. 26, p. 101162, 2024.

[15] M. Aly and M. H. Behiry, "Enhancing anomaly detection in iot-driven factories using logistic boosting, random forest, and svm: A comparative machine learning approach," *Scientific Reports*, vol. 15, no. 1, p. 23694, 2025.

[16] Y. Meidan, M. Bohadana, Y. Mathov, Y. Mirsky, A. Shabtai, D. Breitenbacher, and Y. Elovici, "N-baiot—network-based detection of iot botnet attacks using deep autoencoders," *IEEE Pervasive Computing*, vol. 17, no. 3, p. 12–22, 2018.

[17] N. Moustafa and J. Slay, "Unsw-nb15: a comprehensive data set for network intrusion detection systems (unsw-nb15 network data set)," in *2015 Military Communications and Information Systems Conference (MilCIS)*, pp. 1–6, 2015.

[18] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," 2018.

[19] E. Nunez, E. W. Steyerberg, and J. Nunez, "Regression modeling strategies," *Revista Española de Cardiología (English Edition)*, vol. 64, no. 6, pp. 501–507, 2011.

[20] C. Cortes and V. Vapnik, "Support-vector networks," *Machine learning*, vol. 20, no. 3, pp. 273–297, 1995.

[21] L. Breiman, "Random forests," *Machine learning*, vol. 45, no. 1, pp. 5–32, 2001.

[22] D. Singh and B. Singh, "Feature wise normalization: An effective way of normalizing data," *Pattern Recognition*, vol. 122, p. 108307, 2022.

[23] M. Hossin and M. N. Sulaiman, "A review on evaluation metrics for data classification evaluations," *International journal of data mining & knowledge management process*, vol. 5, no. 2, p. 1, 2015.